College of Computer Studies

DEPARTMENT OF
SOFTWARE TECHNOLOGY

CBINTSY / GDINTAI / CSC613M

Major Course Output 1

📅 Major Details

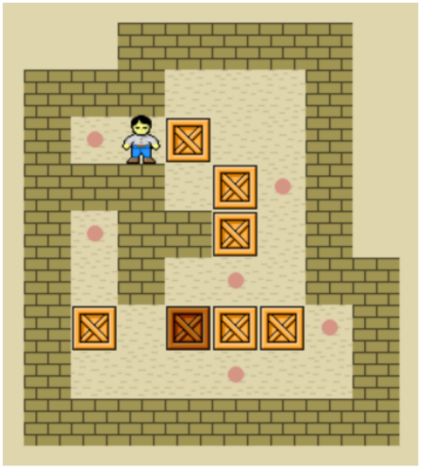
Groupings:	At most 4 members in a group
Deadline:	October 28, 2026 (Wednesday) 6:00 PM (Week 8)
Demo Schedule:	November 5 – 7, 2026 (Week 9)
Percentage:	20%
Submission guidelines:	Submit the `zip` file to AnimoSpace
Filename format:	`Project-<Section>-Group<#>.zip`

✅ Deliverables

`Zip` file containing:

- Python 3 files – `py` files – See Output 1 below
- Report – `pdf` file – See Output 2 below

📌 Specifications



In this project, you will design and implement an artificial intelligence algorithm for a bot that plays **Sokoban**. You must represent the game as a state-based model and apply the necessary techniques and algorithms to automatically solve puzzles in a reasonable time frame.

Sokoban is a classic Japanese puzzle game. It was created by Horiyuki Imabayashi and was first published by Thinking Rabbit in 1981. The name Sokoban (倉庫番) translates to "warehouse keeper". The game has been ported to numerous platforms and has inspired countless unofficial clones.

Sokoban is a grid-based puzzle game where the player organizes crates in a warehouse. The rules of the game are as follows:

1. The player can move in the four orthogonal directions (up, down, left, and right).
2. The player can only stand on empty spaces or target spaces. The player cannot walk through walls.

3. The player can push a crate, only if there is an empty space behind it. The player cannot push multiple crates at once.
4. The player cannot pull crates.
5. The goal of each level is to put all crates on the target spaces.

It is recommended that you try to play a few levels of Sokoban first to get a clear understanding of its rules. A free online implementation of the game can be found in this link.

Despite its simplicity, levels of Sokoban can be very difficult. The difficulty comes from its large branching factor and the fact that solutions in more complex levels can be very deep. Skilled Sokoban players are good in discarding futile and redundant lines of play through the recognition of patterns and subgoals.

Sokoban is a well-studied problem in the field of computer science, as it has implications for real-life applications like motion-planning in robotics. In this project, you will design and implement an algorithm for automatically solving easy Sokoban puzzles.

Sec 1. Starter Program

You are provided a starter program for this project written in Python. Before anything else, you must first test whether the program installs its dependencies and runs successfully on your machine.

For this project, you need Python 3 installed on your machine. If you do not have this yet, there are many tutorials online on how to install Python. Before you proceed, the `python` command should be available from the command line. You also need to install the project's dependencies (namely `pygame`, which is used for the graphical interface) by running the following command once, from the project directory:

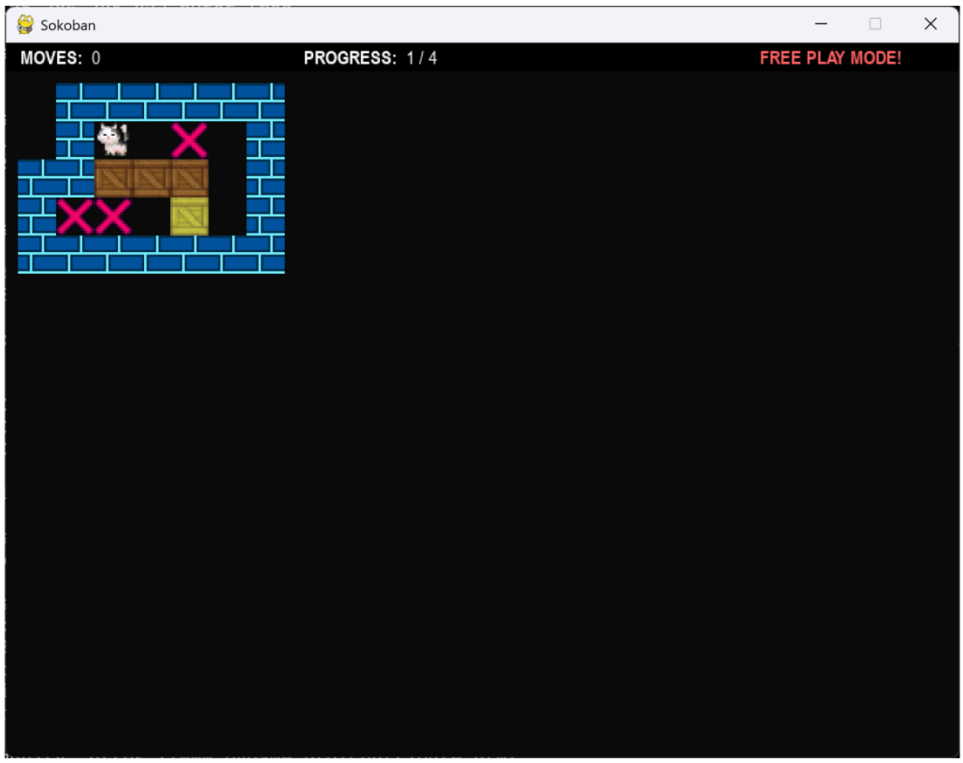
```
pip install -r requirements.txt
```

There are two modes in this program: Free Play Mode and SokoBot Mode. In Free Play mode, you can play Sokoban by yourself using the arrow keys. In SokoBot Mode, the program runs a function to generate a solution to the puzzle. For Windows, you are provided two batch files `freeplay.bat` and `sokobot.bat` to do this easily. You can open the batch files in a text editor to see what commands they use. For UNIX-based systems, `freeplay.sh` and `sokobot.sh` are provided for the same purpose. However, the examples in this document will use the Windows batch files for demonstration.

First, try to run Free Play mode by opening the command line, going into the directory of the project, and invoking the batch file as follows:

```
freeplay testlevel
```

Here, `testlevel` is the name of the level to be loaded. You can replace this with a different level name later if you want. The levels are stored inside the `maps` directory. If successful, you should see a window displaying the level. From here, you can play the game by pressing the arrow keys. You can also press `U` to undo your last move, or `R` to restart the level from its initial state.



Next, try to run SokoBot Mode using the following command:

```
sokobot testlevel
```

By default, this simply prints the bot's resulting solution string to the console, without opening a window. If you want to see an animated playback of the solution instead, add the `--gui` flag:

```
sokobot testlevel --gui
```

Once the window loads, you can press `Space` to start the bot. In the starter program, the bot is completely stupid. It simply thinks (sleeps) for 3 seconds, and then simply moves left and right repeatedly for a number of times. Your task for the project is to rewrite the behavior of this bot to make it intelligent.

If you encounter any problems in running the program on your machine, please consult the instructor for guidance.

Sec 2: Levels and Level Format

Levels should be stored as `.txt` files inside the `maps` directory. The levels should follow the format specified in this link, without any run-length encoding. Additionally, each level should be in its own separate text file. You don't have to worry about writing code to read the files, as the program already does this. Nonetheless, you may want to be familiar with the format if you want to create your own levels for testing. Additionally, you can also download thousands of levels online from various sources. There are many sample levels provided to you along with the starter program.

Output 1: Designing and Implementing SokoBot

Your main task for this project is to design and implement SokoBot, which automatically solves a Sokoban puzzle. To do this, you will change the implementation of the `solveSokobanPuzzle` method in the `SokoBot` class inside the `src/solver` package.

The `solveSokobanPuzzle` accepts four parameters: `width`, `height`, `mapData` and `itemsData`.

- `width` and `height` are integers representing the width and the height of the level.
- `mapData` and `itemsData` are both two-dimensional lists (i.e., lists of lists) of single-character strings representing the layout of the level.
    - `mapData` contains the static elements of the level
    - `itemsData` contains the elements that can move around
    - `mapData` contains '`#`' for a wall, or '`.`' for a target.
    - `itemsData` contains '`@`' for the player, or '`$`' for a crate.
    - All other elements in `mapData` and `itemsData` are set to space characters (' '). Note that it is possible for crates and the player to be in the same location as the targets.

As an example, the level shown in the image above is represented with a `width` of 7, and a `height` of 5, with `mapData` and `itemsData` set up as follows:

```
mapData = [[’ ’, ’#’, ’#’, ’#’, ’#’, ’#’, ’#’],
           [’ ’, ’#’, ’ ’, ’ ’, ’.’, ’ ’, ’#’],
           [’#’, ’#’, ’ ’, ’ ’, ’ ’, ’ ’, ’#’],
           [’#’, ’.’, ’.’, ’ ’, ’.’, ’ ’, ’#’],
           [’#’, ’#’, ’#’, ’#’, ’#’, ’#’, ’#’]]
```

```
itemsData = [[’ ’, ’ ’, ’ ’, ’ ’, ’ ’, ’ ’, ’ ’],
             [’ ’, ’ ’, ’@’, ’ ’, ’ ’, ’ ’, ’ ’],
             [’ ’, ’ ’, ’$’, ’$’, ’$’, ’ ’, ’ ’],
             [’ ’, ’ ’, ’ ’, ’ ’, ’$’, ’ ’, ’ ’],
             [’ ’, ’ ’, ’ ’, ’ ’, ’ ’, ’ ’, ’ ’]]
```

The `solveSokobanPuzzle` method should return a string representing the bot's solution to the given puzzle. The solution string contains the list of moves to be done by the bot, in order. The four directions (i.e., four possible moves) are represented by '`u`', '`d`', '`l`', and '`r`' for up, down, left, and right respectively.

You are allowed to create additional variables, functions, classes, and modules inside the `src/solver` package. However, you are NOT allowed to modify any other codes in the other packages. You are allowed (though not required) to use external libraries for data structures such as trees and graphs (adding them to `requirements.txt` as needed), but you are NOT allowed to use libraries that perform any artificial intelligence algorithms like search.

In addition to solving the puzzle, your bot should also return a solution in a timely manner. To be more specific, the program imposes a 15-second thinking time limit for the bot. If your method does not return a solution within that time frame, the program will refuse to display your solution and will simply display a message that the SokoBot took too long thinking. You must take this time limit into consideration in coming up with your algorithm.

In order to come up with an effective bot in this project, you may need to combine and tweak the algorithms and concepts discussed in class. Simply applying the algorithms naively will result in a poorly-performing bot. Thus, you are encouraged to be creative, critical thinkers throughout the process of this project.

Evaluation and Grading

You are expected to do an evaluation of your bot by testing it on different Sokoban levels. You are encouraged to test it not only on the provided sample cases, but also on other levels as well. As you test the bot, focus not only on whether it was able to solve the levels or not, but also on other aspects such as the number of moves it took to solve the puzzle, the time it took for the solution to be found, among others.

Sokoban has been proven mathematically to be an NP-Hard problem. This means that there is currently no known algorithm that efficiently solves all possible Sokoban levels. However, simpler levels should be solvable. In this project, your bot is not expected to be capable of solving the most complex Sokoban levels, but it must be able to handle as many levels as it can. Your bot will be evaluated on a set of levels that is hidden to you. You can expect the levels for testing to have around 2 to 8 crates. To get full credit, your bot needs to solve at least 80% of the levels that will be used for testing.

Output 2: Report

In addition to the SokoBot codes, you are required to write a report documenting the implementation of the project. The report should contain the following:

1. SokoBot Algorithm: A section describing in detail the algorithm that was used for the implementation of the bot. Focus more on the project-specific details:
    a. The algorithm that was used
    b. How a state was represented in your implementation
    c. How the actions were generated for each state
    d. The rationale behind the implementation of certain search strategies
    e. Description of any heuristics or evaluation functions used in the search strategies
2. Evaluation and Performance: A section explaining the strengths and weaknesses of the bot, as determined by your own testing and evaluation. There must be an empirical evaluation of the bot's performance on a set of different Sokoban levels. There must also be a discussion on the kinds of puzzles can it handle easily, and what kinds of puzzles does it struggle in.
3. Reflections and Challenges: A discussion on the difficulties encountered in developing the bot. Specifically, reflect on the question: what are the challenges that make it difficult for automated solvers to tackle the more complex Sokoban problems?
4. Table of Contributions: A table showing the contributions of each member to the project.

The report should not exceed four pages with A-4 sized paper. If there is a strong need to have more pages, the group must ask permission from the instructor and justify it. There is no restriction on the formatting, as long as it is readable. Please be concise in writing the report and avoid repeating already-known definitions. The bulk of the report should contain the group's personal ideas, insights, and implementations. You are not allowed to use generative AI for the report, even for writing improvements. The report should be an accurate reflection of the group's personal understanding and engagement with the project.

💻 GitHub Repository and Version Control

Create and maintain a GitHub repository for the entire project. All project-related files, including code, documentation, and other relevant materials, should be managed through this repository. Each member of the group is expected to contribute to the repository regularly throughout the project.

The GitHub repository will serve as the sole basis for verifying individual contributions to the project. In particular, GitHub commit history and other repository activity may be examined to:

- Verify the contributions of individual group members, particularly in cases where there are disputes or disagreements regarding the distribution of work.
- Assess whether the project was developed incrementally throughout the designated project period, rather than being completed primarily near the submission deadline.
- Monitor the progress and development of the project across the different weeks of the course.
- Support the assessment and grading of individual contributions to the project.

Students are therefore expected to make regular and meaningful commits that accurately reflect their work and progress. Group members should not rely solely on a final upload of the completed project. Contributions made outside the GitHub repository will not be considered when determining individual contributions and grades.

The GitHub repository and its recorded history will be treated as the sole official reference for evaluating individual project contributions. Students are responsible for ensuring that their work is properly committed and that their contributions can be clearly identified in the repository.

📖 Project Oral Exam

Here are some guidelines regarding the project oral exam:

- Each group is given 30 minutes for Q&A.
- Presentations will be done either online or face-to-face.
- Your instructor will open your notebook and will ask questions about your project.
- All members should be present during the oral exam.
- Kindly read the specifications to check different requirements and expectations on the project.
- Failure to demonstrate ownership and genuine understanding during the oral exam may result in a grade of 0 for the group.

🏋️ Working With Groupmates

Make sure that each member of the group has approximately the same amount of contribution for the project. Problems with groupmates must be discussed internally within the group, and if needed, with the lecturer.

🧠 Use of Artificial Intelligence Tools

You are allowed to use AI tools to assist you in the creation of your work, under the following conditions:

1. You must declare the use of such tools in your report.
2. Do not generate any code or code snippets. Students must implement algorithms from scratch to gain a deep understanding of their components.
3. Do not use these tools for any form of text generation or writing improvements in any submitted work. Evaluations are focused on the students' technical and conceptual understanding and not their writing styles.
4. You must be able to articulate the thought processes, rationales, and implementation details of your work, and through this you must be able to show that human agency was maintained even if AI was used in augmenting the process.

Use of AI outside of these parameters is considered academic dishonesty.

🎓 Academic Honesty Policy

Honesty policy applies. Please take note that you are NOT allowed to borrow and/or copy-and-paste – in full or in part – any existing related program code or solutions from the internet or other sources (such as printed materials like books, or source codes by other people that are not online). You should develop your own codes and solutions from scratch by yourselves.

The student handbook states that (Sec. 5.2.4.2):

"Faculty members have the right to demand the presentation of a student's ID, to give a grade of 0.0, and to deny admission to class of any student caught cheating under Sec. 5.3.1.1 to Sec. 5.3.1.1.6. The student should immediately be informed of his/her grade and barred from further attending his/her classes."

The student handbook also states that (Sec. 10.3):

A student caught cheating, as defined in Sec. 5.3.1.1., shall be penalized with a grade of 0.0 in the requirement or in the course, at the discretion of the faculty member, without prejudice to an administrative sanction. In cases of alleged cheating, the faculty member should report the incident to the Student Discipline Formation Office (SDFO).